hangar: a lightweight module system for underwater vehicles, with a recorded learning loop

(Research proposal)

H. Oğuz Bolat
Department of Geomatics Engineering
Yıldız Technical University
İstanbul, Türkiye
oguz.bolat@std.yildiz.edu.tr

***Abstract*—hangar is a module system for remotely operated vehicles (ROVs) of the BlueROV2 class, native to Linux and small enough for the vehicle's Raspberry Pi. Each function of the vehicle is its own process under one manager; processes exchange fixed-size messages through shared memory, and no reader can delay a writer. The learning loop is part of this infrastructure, not an application on top of it. Every message records, by identifier, the input messages it used. One dataset definition produces the same model inputs live, offline and inside the running policy. Weights trained on a topside GPU are checked by the vehicle before use, run in shadow until an operator promotes the evaluated model by its hash, and are demoted automatically. Any thruster command can then be traced, by identifiers alone, to the model that produced it, the rows that trained it, and the sensor readings and pilot commands behind those rows. hangar supports known deployment-time learning schemes (pilot takeovers as in HG-DAgger [1] and Sirius [2]; a residual on a classical controller) and makes them recorded, checked and reversible on a Pi. I propose to implement it in C11, compare it with ROS 2 on the same Pi, test its failure behaviour on the bench, and measure what the loop costs and what its checks catch.**

***Index Terms*—remotely operated vehicles, module system, shared memory, provenance, deployment-time learning, Raspberry Pi**

I. Motivation and problem

On every dive the pilot shows how the vehicle should be driven. Learning from this data meets three obstacles.

A recording holds messages and timestamps, not which inputs a module used for each output, so training pairs are rebuilt by matching times, and features built offline can differ from what the model meets on the vehicle. Different code paths for training and serving are the most common cause of this skew [3]: LeRobot trains on decoded video and deploys on raw frames [4]; AquaBot trains on JPEG frames and deploys on uncompressed ones, with a time join that can look forward in time [5]. Message-level provenance exists for ROS, carried in headers [6] or reconstructed from traces [7], but it has not been used to build datasets.

Moving a model trained topside onto a running vehicle is, in the systems we know of, a manual step outside the middleware, with nothing on the vehicle to check the weights or remove a misbehaving model.

ROS 2 brings a DDS participant per process and quadratic discovery to a computer that also runs the autopilot and encodes video; its authors measure 2.1 to 4.3 MB per added process and recommend composition into one process, which gives up isolation [8].

II. Proposed approach

Transport and supervision. A topic is one shared-memory ring per output, with one writer and any number of readers; a sequence counter per slot, read in Lamport's order [9], detects torn copies. The ring itself is old; iceoryx2 is a current example [10]. What differs: readers hold no resources, so a dead or stuck reader needs no cleanup and delays nobody, and schema, provenance and a staleness flag live in the ring file. The manager restarts a failed module alone, with back-off, and moves no data; if it crashes or is re-executed for an upgrade, the modules keep running and are adopted again from a small registry. This is crash-only design [11] and one-for-one supervision [12]; the recovery from a registry is ROS Rescue's, for the ROS 1 master [13].

Provenance and datasets. Each message lists up to four input messages as (topic, sequence number), filled in by the library at publish time; an output computed from a stale input is flagged, transitively. This is recorded usage, not proven derivation [14]. A dataset spec names an anchor topic and the inputs joined to it, by provenance or by a bounded time join. The same join code runs in the live pairer on the vehicle, in the offline build, and in the policy that builds its inputs. Live and offline rows are identical except where a `late` flag marks a message that arrived after the live row was emitted. A bridge mirrors topics over the tether with their sequence numbers, so a provenance link means the same message on both sides, and pilot commands take priority over dataset rows and weight files.

Policies. A policy module runs a small inference engine that allocates all memory at load. The engine is standard engineering; the handover is not. A weights file carries an architecture hash, probe inputs with expected outputs and tolerances, and the lineage of its training rows. The vehicle checks hashes, finite values, probes and step time against its tick budget, warms the model up beside the running one and swaps between two ticks; `revert` restores the previous one. The probe check is differential testing in CRADLE's sense [15], done by the receiving vehicle rather than offline.

Safety. All control sources pass through one multiplexer (the mux) that enforces thrust, rate and depth limits per axis; the pilot wins on any axis. The mux is a Simplex-style switch [16] with a hand-written envelope and a latched demotion that only a human releases [17]; SOTER, the closest precedent, switches back automatically where hangar waits for an operator [18]. A new model runs in shadow until promoted, and is demoted if its output goes stale or sits at a limit for too long. One gate per actuator outputs neutral unless the vehicle is armed and the command is fresh. The envelope bounds commands, not states; it is not a safety proof. In phase 1 ArduSub still drives the thrusters, held in MANUAL mode with its failsafes on, and hangar owns the ground-station link. Version 1 of the loop drives only in residual mode, a bounded correction added to a classical controller [19], as Neural-Fly did beside an autopilot on a Pi 4 [20].

III. Contribution and prior work

A survey of about 110 works preceded this revision. Every part of hangar has prior art; the claim is the combination and where it is done. To our knowledge, no system combines the properties above, and none on an embedded vehicle behind a tether. hangar claims no new learning method, safety method or loop; Table I names the closest prior work. The secondary claims, a smaller footprint than ROS 2 with process isolation kept and safe failure behaviour, are not new in kind and are reported as measured.

IV. Evaluation plan

Table II lists eight measurements that no surveyed work reports and that carry the claim. Each is a tool in the repository, reproducible by a script. Where the design states a budget, that is the pass criterion; the rest are reported as measured.

Secondary: a fault campaign with propellers off (SIGKILL per module, 100 runs; time to `ready`; time to neutral at the PWM output; manager crash and upgrade with calls in flight); pilot overrides per promoted model; residual adaptation after a mass change, in water; and the electric-eye port hashed against v0.3.1, with video latency no worse than 3.8 ms at 640×480 and 8.3 ms at 1280×720. The learning tasks run on recorded data, then in simulation and on the bench, and in water only when a vehicle and a tank are available.

V. Work plan (draft, to be adjusted)

1) **Language and literature.** Done; see Status.
2) **Core.** Topics, manager, calls, parameters; budgets as tests; the benchmark on a Pi.
3) **Vehicle integration.** Safety, mux and gate; ArduSub in simulation; tether bridge; electric-eye supervised unchanged.
4) **Data.** Recorder, pairer, offline build; the live-equals-offline test.
5) **Learning loop.** Inference engine, weights file, lineage; both tasks on recorded data.
6) **Bench and water.** Fault campaign; both tasks on the bench, then in water if possible; electric-eye ported and hashed.
7) **Write-up.**

VI. Risks

- **Scope and hardware.** Fifteen design documents for one student; each phase ends in a measurable result, and the loop can be shown in simulation and on the bench without wet runs. The work needs a Pi 4 or 5, a topside computer with a GPU and, for wet runs, a vehicle and a tank.
- **The depth-hold example is wrong as designed.** It learns total heave and runs as a residual, so it would sit at its cap. Its label, the heave stick command, is not heave thrust: ArduSub's stock mixer scales all outputs down against the largest input to avoid saturation [29], so the command is not thrust once other axes are active. Label and cap are being reworked; the cap becomes a quantity measured on the vehicle.
- **Narrowed claims.** After the survey and two adversarial reviews, "byte-identical rows" became "identical except where flagged late"; "computed from" became "used"; pilot imitation is HG-DAgger, not DAgger; the envelope gives no state guarantee; and version 1 drives only in residual mode. Three related works and the full claims of the Ford patents are still unread, so the claim may narrow further.
- **Weak policies and footprint.** A few dives may not train a good policy, and the survey found no precedent for a small network on a Pi CPU imitating a pilot from camera frames at 50 Hz. If hangar is not much lighter than composed ROS 2, that is reported.

VII. Current status

The design is complete in fifteen documents (00–14), including deployment (signed releases, upgrade over the tether) and simulation (two managers on one machine, a 6-DOF model, a camera fed from recorded footage, scripted faults), revised after two adversarial reviews and a simplification pass. The language is decided: C11, static musl binaries built with zig cc. A spike built the same topic and manager slice in C and in Rust, on x86-64 only: the C manager is 50 KB static against 435 KB, idle resident memory 136 KB against 480 KB, publish 29 ns against 35 ns; wake latency is 5.4 µs at the median for both, and both showed 0 torn reads over 250 million and 0 lost wake-ups. Nothing has run on a Pi. electric-eye v0.3.1 [30], whose frame bus, process handling, discovery and configuration hangar builds on, is forked into the repository and builds with zig cc natively and as static AArch64 musl binaries with NEON; its 17 test suites pass natively and under qemu-aarch64. OpenMP is unavailable in static musl builds, so its operator loops will use a thread pool when ported. Licences: GPL-3.0-or-later for the system, LGPL-3.0-or-later for the module library and the Python package. Nothing has run on a vehicle.

TABLE I
CLOSEST PRIOR WORK

Work	What it has; what hangar adds, or merely combines
SERL and HIL-SERL [21], [22]; LeRobot [4]	The loop's shape is theirs (actor on the robot, learner on a GPU, weights pushed back while the robot runs, human takeover in the actor); hangar adds checks on received weights, shadow, promotion, demotion and lineage.
Sirius [2]	The scheme is Sirius's and HG-DAgger's [1] (per-sample "who acted" labels, retraining in rounds during deployment); hangar records the per-axis source in the data plane and makes rows, weights and promotions traceable.
Neural Simplex [23]	Shadow mode, retraining while the baseline drives and automatic switch-back, in simulation; hangar runs shadow on a real vehicle with an operator, not a test, in the switch.
Ford patents [24], [25]	A newly trained model run in shadow on the vehicle and validated against the previous one; a method, not a measured system, and as read without hash-bound operator promotion, probe or step-time checks, automatic demotion or lineage.
TFX [26], ML Metadata [27]	One transform for training and serving, "safe to serve" checks and artefact lineage, on servers; hangar moves the checks onto the vehicle and extends lineage from artefacts to messages and actuator commands.
Witte and Tichy [6]; Bédard et al. [7]	Message-level provenance in ROS, per field in headers or inferred from traces; hangar fills fixed-size identifier references at publish time, keeps them across the tether and builds datasets from them.
AquaBot [5]	Pilot demonstrations on a low-cost ROV, behaviour cloning and improvement from rollouts, with the policy topside on two GPUs; hangar runs the policy on the vehicle under a gate without AquaBot's train/serve skew, and is not the first to learn from an ROV pilot.
Carlucho et al. [28]	Online deep reinforcement learning on a real AUV with full authority to the network; hangar bounds that authority and records the loop, the supervisor layer they list as future work.

TABLE II
MEASUREMENTS THAT CARRY THE CLAIM

Quantity	How it is measured
Train/serve mismatch rate, per dive	Each policy publishes (anchor sequence, CRC of its input) per tick; key-joined against the built rows; share of ticks that differ
Provenance overhead on a Pi	Publish latency and memory with 0 and with 4 references, Pi 4 and Pi 5
Loop latency	Newest row a model trained on to its first promoted output, on one clock
Cost of a hot swap	Tick jitter and missed ticks before, during and after a weights load at 50 Hz
What the vehicle-side checks catch	Probe, tolerance and step-time refusals over mutated weights files (wrong operator semantics, precision change, corrupted weights)
Demotion and envelope-activation rates	Per dive and per model, from the mux state and the evaluation topic; false demotions listed
Lineage completeness	Share of thruster commands traceable to messages; forward queries from a bad message to the models it reached
Isolation at what cost	Memory (PSS) and latency of N isolated modules against composed ROS 2 [8] and iceoryx2 [10], same Pi 4, message sizes and rates

REFERENCES

[1] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "HG-DAgger: Interactive imitation learning with human experts," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 8077–8083.

[2] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu, "Robot learning on the job: Human-in-the-loop autonomy and learning during deployment," in *Robotics: Science and Systems XIX*, 2023, extended version in Int. J. Robotics Research 44(10–11), 2025.

[3] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proceedings of Machine Learning and Systems (MLSys/SysML 2019)*, 2019. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/928f1160e52192e3e0017fb63ab65391-Abstract.html

[4] R. Cadene, S. Aliberts, F. Capuano, M. Aractingi, A. Zouitine, P. Kooijmans, J. Choghari, M. Russi, C. Pascal, S. Palma, M. Shukor, J. Moss, A. Soare, D. Aubakirova, Q. Lhoest, Q. Gallouédec, and T. Wolf, "LeRobot: An open-source library for end-to-end robot learning," arXiv:2602.22818, 2026.

[5] R. Liu, H. Ha, M. Hou, S. Song, and C. Vondrick, "Self-improving autonomous underwater manipulation," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 16 915–16 922.

[6] T. Witte and M. Tichy, "Inferred interactive controls through provenance tracking of ROS message data," in *2021 IEEE/ACM 3rd International Workshop on Robotics Software Engineering (RoSE)*, 2021, pp. 67–74.

[7] C. Bédard, P.-Y. Lajoie, G. Beltrame, and M. Dagenais, "Message flow analysis with complex causal links for distributed ROS 2 systems," *Robotics and Autonomous Systems*, vol. 161, p. 104361, 2023.

[8] S. Macenski, A. Soragna, M. Carroll, and Z. Ge, "Impact of ROS 2 node composition in robotic systems," *IEEE Robotics and Automation Letters*, vol. 8, no. 7, pp. 3996–4003, 2023.

[9] L. Lamport, "Concurrent reading and writing," *Communications of the ACM*, vol. 20, no. 11, pp. 806–811, 1977.

[10] Eclipse iceoryx contributors, "Eclipse iceoryx2: the true zero-copy data plane," GitHub repository, 2026, repository, commit 69973dd. [Online]. Available: https://github.com/eclipse-iceoryx/iceoryx2

[11] G. Candea and A. Fox, "Crash-only software," in *9th Workshop on Hot Topics in Operating Systems (HotOS IX)*. USENIX, 2003. [Online]. Available: https://www.usenix.org/conference/hotos-ix/crash-only-software

[12] J. Armstrong, "Making reliable distributed systems in the presence of software errors," Ph.D. dissertation, The Royal Institute of Technology (KTH), Stockholm, Sweden, Dec. 2003.

[13] P. Kaveti and H. Singh, "ROS rescue: Fault tolerance system for robot operating system," arXiv:1910.01078, 2019.

[14] L. Moreau and P. Missier, Eds., "PROV-DM: The PROV data model," World Wide Web Consortium (W3C), W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/2013/REC-prov-dm-20130430/

[15] H. V. Pham, T. Lutellier, W. Qi, and L. Tan, "CRADLE: Cross-backend validation to detect and localize bugs in deep learning libraries," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 1027–1038.

[16] D. Seto, B. Krogh, L. Sha, and A. Chutinan, "The Simplex architecture

for safe online control system upgrades," in *Proceedings of the 1998 American Control Conference (ACC)*, vol. 6, 1998, pp. 3504–3508.

[17] K. Hobbs, M. Mote, M. Abate, S. Coogan, and E. Feron, "Run time assurance for safety-critical systems: An introduction to safety filtering approaches for complex control systems," *IEEE Control Systems*, vol. 43, no. 2, pp. 28–65, 2023.

[18] A. Desai, S. Ghosh, S. A. Seshia, N. Shankar, and A. Tiwari, "SOTER: A runtime assurance framework for programming safe robotics systems," in *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2019, pp. 138–150.

[19] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 6023–6029.

[20] M. O'Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, "Neural-fly enables rapid learning for agile flight in strong winds," *Science Robotics*, vol. 7, no. 66, p. eabm6597, 2022.

[21] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine, "SERL: A software suite for sample-efficient robotic reinforcement learning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 16 961–16 969.

[22] J. Luo, C. Xu, J. Wu, and S. Levine, "Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning," *Science Robotics*, vol. 10, no. 105, p. eads5033, 2025.

[23] D. T. Phan, R. Grosu, N. Jansen, N. Paoletti, S. A. Smolka, and S. D. Stoller, "Neural simplex architecture," arXiv:1908.00528, 2019, 12th NASA Formal Methods Symposium (NFM 2020).

[24] S. Singh, H. B. Bhate, Z. Rawashdeh, U. Thakre, V. Zavadsky, and S. R. Maram, "Automatic onboard validation of a newly trained vehicle machine learning model," US Patent 12,468,964 B2, Nov. 11, 2025, assignee: Ford Global Technologies, LLC. [Online]. Available: https://patents.google.com/patent/US12468964B2/en

[25] U. Thakre, H. B. Bhate, S. Singh, Z. Rawashdeh, Z. Zeng, S. K. Natarajan, V. Zavadsky, S. Janardhanan, and S. V. Murugesan, "Online in-vehicle learning for machine learning models," US Patent 12,561,586 B2, Feb. 24, 2026, assignee: Ford Global Technologies, LLC. [Online]. Available: https://patents.google.com/patent/US12561586B2/en

[26] D. Baylor, E. Breck, H.-T. Cheng, N. Fiedel, C. Y. Foo, Z. Haque, S. Haykal, M. Ispir, V. Jain, L. Koc, C. Y. Koo, L. Lew, C. Mewald, A. N. Modi, N. Polyzotis, S. Ramesh, S. Roy, S. E. Whang, M. Wicke, J. Wilkiewicz, X. Zhang, and M. Zinkevich, "TFX: A TensorFlow-based production-scale machine learning platform," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 2017, pp. 1387–1395.

[27] Google, "ML Metadata (MLMD): For recording and retrieving metadata associated with ML developer and data scientist workflows," GitHub repository, 2026, documentation, read 2026-11-07. [Online]. Available: https://github.com/google/ml-metadata

[28] I. Carlucho, M. De Paula, S. Wang, Y. Petillot, and G. G. Acosta, "Adaptive low-level control of autonomous underwater vehicles using deep reinforcement learning," *Robotics and Autonomous Systems*, vol. 107, pp. 71–86, 2018.

[29] P. Ng and M. Krieg, "Modifications to ArduSub that improve BlueROV SITL accuracy and design of hybrid autopilot," *Applied Sciences*, vol. 14, no. 17, p. 7453, 2024.

[30] H. O. Bolat and H. Ersoy, "electric-eye: Real-time image enhancement and low-latency video transport for remotely operated underwater vehicles," manuscript describing electric-eye v0.3.1.